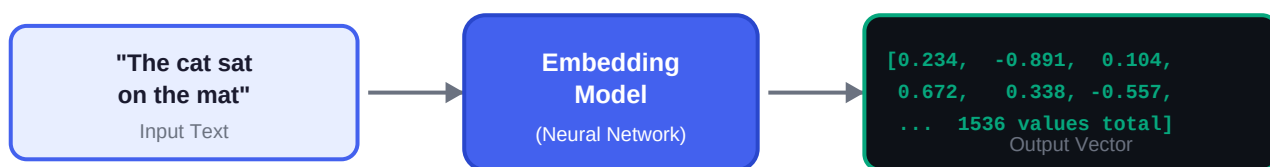


Understanding Embeddings

Turning Text into Meaning-Preserving Vectors



A beginner-friendly guide with diagrams and live Python code examples using the **OpenAI** and **Anthropic** APIs.

Contents

1. What is an Embedding?
2. From Text to Numbers — How it Works
3. Vector Space & Semantic Similarity
4. Cosine Similarity
5. OpenAI Embeddings API (Python)
6. Anthropic Embeddings (Python)
7. Practical Use Cases
8. Choosing an Embedding Model
9. Quick Reference Cheat Sheet

1. What is an Embedding?

Computers cannot understand words directly — they only understand numbers. An **embedding** is a way to convert a piece of text (a word, sentence, or even a whole document) into a **list of numbers** (called a **vector**) such that the **meaning of the text is captured** in those numbers.

■ Key Insight: Similar meanings → similar numbers. "cat" and "kitten" will produce vectors that are close together in space, while "cat" and "pizza" will be far apart.

Think of it as placing every piece of text on a giant map. Words or sentences with similar meanings end up near each other on the map, while unrelated concepts end up far apart. This map lives in a very high-dimensional space (hundreds or thousands of dimensions), but the idea is exactly the same as a regular 2-D map.

An analogy

Imagine you have a huge library. You could organise books randomly — that makes finding related books impossible. Or you could organise them by topic so that books on similar subjects sit on nearby shelves. Embeddings do exactly this, but for text, and in a mathematically precise way.

2. From Text to Numbers — How it Works

The transformation from raw text to a vector happens inside a neural network that has been trained on enormous amounts of text. Here is the pipeline:

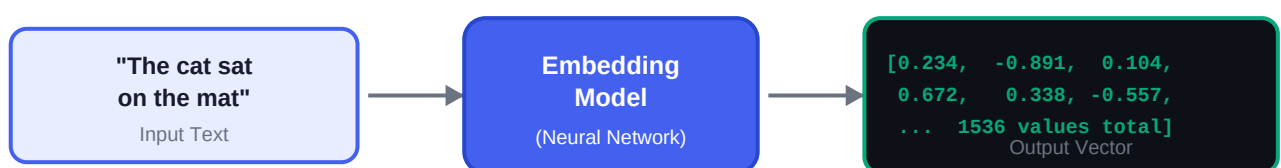


Figure 1 — A sentence enters the embedding model and exits as a fixed-length vector of numbers.

Step-by-step

- **Tokenisation** — The text is split into tokens (roughly words or sub-words). e.g. "running" → ["run", "##ning"]
- **Token Embeddings** — Each token is converted to a small initial vector using a lookup table learned during training.
- **Transformer Layers** — The model processes all tokens together, letting each token 'see' all others. This is how context is captured: 'bank' in 'river bank' gets a different vector from 'bank' in 'savings bank'.

- **Pooling** — The per-token vectors are combined (e.g. averaged) into one single fixed-size vector representing the whole input.

■ The output vector is always the same length regardless of input length. OpenAI's text-embedding-3-small produces 1 536-dimensional vectors. That means every piece of text — one word or ten paragraphs — becomes a list of exactly 1 536 numbers.

3. Vector Space & Semantic Similarity

Because every text becomes a point in the same high-dimensional space, we can compare any two texts by measuring the **distance** or **angle** between their vectors. The chart below shows a toy 2-D projection — in reality there are hundreds of dimensions.



Figure 2 — Related words cluster together. Animal words group near each other, food words form a separate cluster, and tech terms form their own region.

This clustering is entirely automatic — the model learned it from reading billions of sentences. Nobody hard-coded the rule 'cats and dogs are similar'. The model discovered it because those words appear in similar contexts.

4. Cosine Similarity

The most common way to compare two embeddings is **cosine similarity**. Instead of measuring straight-line distance, it measures the **angle** between the two vectors — a small angle means similar meaning, a large angle means different meaning.

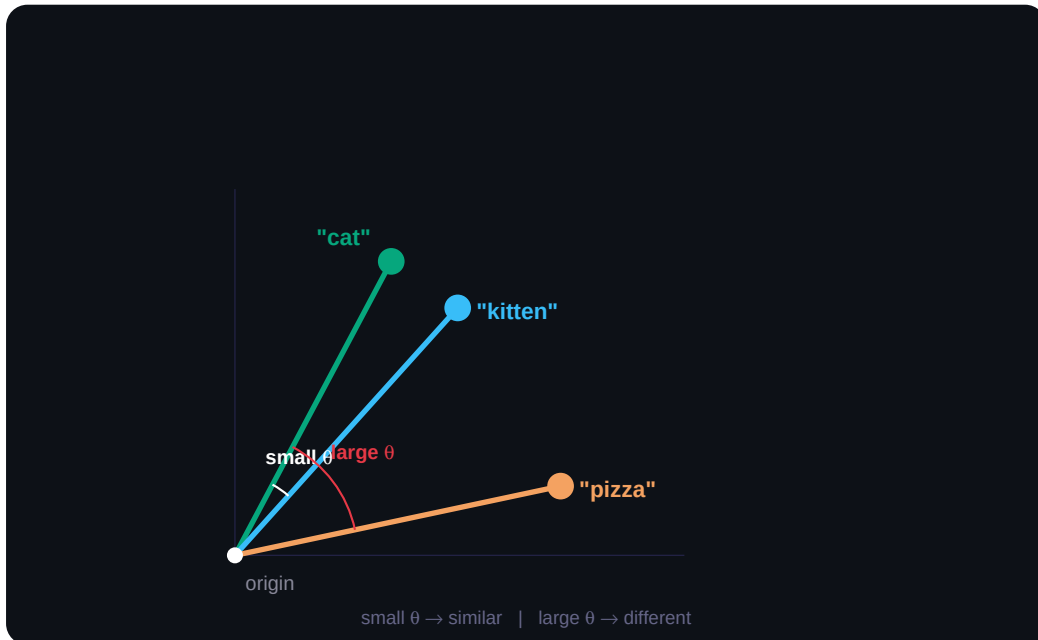


Figure 3 — 'cat' and 'kitten' have a small angle (similar), while 'cat' and 'pizza' have a large angle (unrelated).

The Formula

$$\cos(\theta) = (A \cdot B) / (|A| \times |B|)$$

Term	Meaning
$A \cdot B$	Dot product of the two vectors
$ A , B $	Lengths (magnitudes) of each vector
Result = 1.0	Identical meaning (0° angle)
Result = 0.0	Unrelated (90° angle)
Result = -1.0	Opposite meaning (180° angle)

5. OpenAI Embeddings API (Python)

OpenAI text-embedding-3-small | text-embedding-3-large | text-embedding-ada-002

Installation

```
pip install openai
```

Getting a single embedding

```
"""OpenAI Embeddings - minimal example"""

from openai import OpenAI

client = OpenAI(api_key="YOUR_OPENAI_API_KEY")

# Generate an embedding for a sentence

response = client.embeddings.create(

    input="The cat sat on the mat",

    model="text-embedding-3-small"

)

vector = response.data[0].embedding

print(f"Dimensions: {len(vector)}") # → 1536

print(f"First 5 values: {vector[:5]}")
```

Computing cosine similarity between two texts

```
import numpy as np

from openai import OpenAI

client = OpenAI(api_key="YOUR_OPENAI_API_KEY")

def get_embedding(text: str) -> list[float]:

    resp = client.embeddings.create(

        input=text, model="text-embedding-3-small")

    return resp.data[0].embedding

def cosine_similarity(a, b) -> float:

    a, b = np.array(a), np.array(b)

    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

# Compare three sentences

v_cat = get_embedding("I love my cat")
```

```
v_kitten = get_embedding("My kitten is adorable")

v_pizza = get_embedding("I want to order a pizza")

print(cosine_similarity(v_cat, v_kitten)) # → ~0.92 (similar)

print(cosine_similarity(v_cat, v_pizza)) # → ~0.28 (different)
```

Batch embedding (multiple texts at once)

```
texts = [

    "Machine learning is fascinating",
    "Deep learning uses neural networks",
    "I enjoy cooking Italian food",
]

response = client.embeddings.create(

    input=texts,

    model="text-embedding-3-small"

)

# Each item in response.data corresponds to one input text

vectors = [item.embedding for item in response.data]

print(f"Got {len(vectors)} embeddings, each of size {len(vectors[0])}")
```

■ Cost tip: Use text-embedding-3-small for most tasks — it is 5× cheaper than text-embedding-3-large and performs excellently for semantic search and retrieval. Switch to large only when precision is critical.

6. Anthropic Embeddings (Python)

Anthropic voyage-3 | voyage-3-lite | voyage-3-large | voyage-code-3

Anthropic partners with **Voyage AI** for embeddings. Voyage models are among the highest-ranked models on the MTEB (Massive Text Embedding Benchmark) leaderboard. The API is simple and the Python client is lightweight.

Installation

```
pip install voyageai
```

Getting a single embedding

```
"""Anthropic / Voyage AI Embeddings — minimal example"""

import voyageai

vo = voyageai.Client(api_key="YOUR_VOYAGE_API_KEY")

# Embed a single sentence

result = vo.embed(

    ["The cat sat on the mat"],

    model="voyage-3",

    input_type="document"

)

vector = result.embeddings[0]

print(f"Dimensions: {len(vector)}") # → 1024
```

input_type — an important parameter

Voyage models are optimised separately for queries versus documents. Always set **input_type** correctly for best results:

- **"query"** — short search query from the user (e.g. "best pasta recipe")
- **"document"** — longer content you want to store and search (e.g. a recipe article)

Semantic search example with Voyage

```
import voyageai
```

```
import numpy as np

vo = voyageai.Client(api_key="YOUR_VOYAGE_API_KEY")

# A small knowledge base (documents to search)

documents = [

    "Photosynthesis converts sunlight into glucose in plants",

    "The Eiffel Tower is located in Paris, France",

    "Neural networks learn from labelled examples",

    "Mitochondria produce ATP through cellular respiration",

]

# Embed all documents once and store them

doc_result = vo.embed(documents, model="voyage-3",

    input_type="document")

doc_vectors = np.array(doc_result.embeddings)

# At search time: embed the user query

query = "How do plants make food?"

q_result = vo.embed([query], model="voyage-3",

    input_type="query")

q_vec = np.array(q_result.embeddings[0])

# Cosine similarity – find the most relevant document

scores = doc_vectors @ q_vec / (

    np.linalg.norm(doc_vectors, axis=1) * np.linalg.norm(q_vec)

)

best_idx = int(np.argmax(scores))

print(f"Best match ({scores[best_idx]:.3f}): {documents[best_idx]}")

# → Best match (0.861): Photosynthesis converts sunlight...
```


■ Where to get a Voyage API key: Sign up at voyageai.com — Anthropic's documentation directs users there for embeddings. Your Anthropic API key is NOT the same as your Voyage key.

7. Practical Use Cases

■ Semantic Search

Instead of matching keywords, find documents by meaning. "What causes rain?" matches "precipitation formation" even with no shared words.

■ RAG (Retrieval-Augmented Generation)

Embed your documents, store vectors in a vector database (Pinecone, Chroma, Weaviate), then at query time retrieve the most relevant chunks and feed them to an LLM. This gives the LLM accurate, up-to-date context.

■ Classification & Clustering

Group similar customer support tickets, news articles, or emails automatically by clustering their embeddings — no labelled training data required.

■ Deduplication

Detect near-duplicate content by finding pairs of embeddings with very high cosine similarity (e.g. > 0.97).

■ Recommendation Systems

Recommend products, articles, or songs by finding items whose descriptions are semantically similar to what the user has previously liked.

■ Cross-lingual Search

Multilingual models embed 'gato' (Spanish) and 'cat' (English) near each other, enabling search across languages with no translation step.

The RAG pipeline at a glance

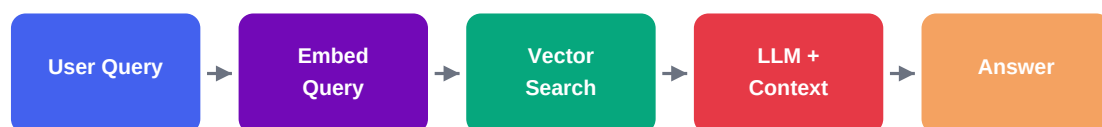


Figure 4 — A Retrieval-Augmented Generation pipeline. The query is embedded, matched against stored document vectors, then the top results are fed to the LLM alongside the question.

8. Choosing an Embedding Model

text-embedding-3-small	OpenAI	1 536	Cost-effective search, RAG	semantic

text-embedding-3-large	OpenAI	3 072	High-precision tasks, re-ranking
text-embedding-ada-002	OpenAI (legacy)	1 536	Legacy systems — prefer 3-small instead
voyage-3	Voyage / Anthropic	1 024	General purpose, top MTEB scores
voyage-3-large	Voyage / Anthropic	1 024	Highest accuracy retrieval tasks
voyage-3-lite	Voyage / Anthropic	512	Low latency, cost-sensitive workloads
voyage-code-3	Voyage / Anthropic	1 024	Code search and retrieval

Table 1 — Common embedding models. All are accessible via simple REST/Python APIs.

9. Quick Reference Cheat Sheet

Concept	One-line explanation
Embedding	Fixed-length vector of numbers that encodes the meaning of text
Dimension	Number of values in the vector (e.g. 1536 for text-embedding-3-small)
Cosine similarity	Angle-based score in $[-1, 1]$; 1 = identical meaning
Vector database	Specialised DB that stores & searches embeddings efficiently (Pinecone, Chroma, FAISS)
RAG	Retrieve relevant documents via embeddings, then pass them to an LLM
input_type	Voyage param: 'query' for user searches, 'document' for stored content
Tokenisation	Splitting text into sub-word tokens before embedding
Pooling	Combining per-token vectors into one sentence-level vector
MTEB	Benchmark for ranking embedding models across dozens of tasks

Key Takeaways

- An **embedding** translates text into a vector where **similar meanings produce similar vectors**.
- The transformation is done by a **pre-trained neural network** — you do not need to train one yourself.
- Use **cosine similarity** to compare embeddings: scores close to 1 mean the texts are semantically related.
- The **OpenAI API** (*text-embedding-3-small*) and **Voyage AI / Anthropic** (*voyage-3*) both offer high-quality embeddings with a few lines of Python.
- Always distinguish **query** embeddings (short user input) from **document** embeddings (content to search) when using Voyage models.
- Embeddings power **semantic search, RAG, clustering, deduplication**, and many other AI applications.

Happy embedding! — Now try building a small semantic search tool using the code examples in this guide.